

T14 — Classi Annidate, Interne, Locali ed Anonime

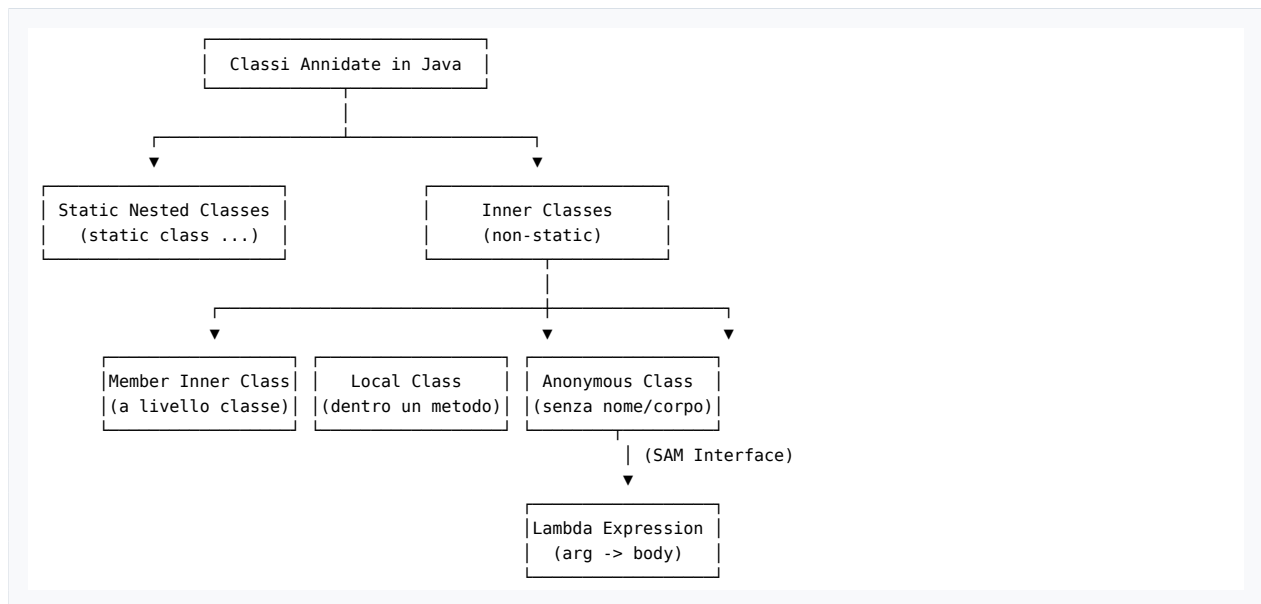
Static Nested vs Inner Classes, Enclosing Pointer, Shadowing, Effectively Final e Classi Anonime

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

0.1 1. Analisi Teorica Approfondita (Slide T14)

La lezione 14 esplora la gerarchia delle classi definite all'interno di altre classi (meccanismo introdotto fin da Java 1.1), le classi anonime, l'evoluzione verso le **espressioni Lambda** (Java 8) e la feature dei **Varargs**.



0.1.1 1.1 Static Nested Classes (static class Nested)

Una classe dichiarata con il modificatore `static` all'interno di un'altra classe contenitore (*enclosing class*):

```
1 package pkg;
2 class Outer {
3     static class Nested {
4         // ...
5     }
6 }
```

- **Natura:** È a tutti gli effetti una classe di primo livello (*top-level*), ma inserita nel *namespace* della classe che la ospita.
- **Nome completo (Fully Qualified Name):** `pkg.Outer.Nested` (a livello di bytecode su disco viene generato il file `Outer$Nested.class`).
- **Accesso ai membri:**
 - Trattandosi di un membro statico, **ha accesso a tutti i membri statici** della classe contenitore, **compresi quelli privati (`private`)**!
 - **NON ha accesso** ai membri di istanza (campi o metodi non statici) di `Outer`, poiché non esiste alcun riferimento implicito ad un'istanza di `Outer`.
- **Istanziamento:** Può essere istanziata in modo totalmente indipendente dall'esistenza di un oggetto `Outer`:

```
1 Outer.Nested nestedObject = new Outer.Nested();
```

- **Caso d'uso tipico:** Il pattern **Builder** (visto in [Date.java](#)), classi di utilità, nodi interni di strutture dati (es. `static class Node<E>` in un albero o grafo).

0.1.2 1.2 Member Inner Classes (`class` Inner non-statica)

Una classe dichiarata all'interno di un'altra classe **senza** la parola chiave `static`:

```
1 package pkg;
2 class Outer {
3     class Inner {
4         // ...
5     }
6 }
```

- **Natura:** Un'istanza di `Inner` **non può esistere senza un'istanza associata di `Outer`**.
- **Riferimento implicito (`Outer.this` o `this$0`):**
 - Ogni oggetto `Inner` memorizza internamente un puntatore sintetico nascosto all'oggetto `Outer` che l'ha generato.
 - Tramite questo puntatore, `Inner` ha accesso a **tutti** i membri di `Outer`, sia statici che di istanza, inclusi quelli `private`!
- **Sintassi di istanziamento da codice esterno:**

```
1 Outer outerObject = new Outer();
2 Outer.Inner innerObject = outerObject.new Inner(); // Sintassi specifica con outerObject.new
```

Confronto: Nested vs Inner (Slide 7)

```
1 class Outer {
2     private static int I = 5;
3     private int i;
4
5     static class Nested {
6         // NON ha accesso al campo di istanza i!
7         static void print() { System.out.println(I); } // OK: accede a membro statico privato I
8     }
9 }
```

```

10  class Inner {
11      // Ha accesso sia a I che al campo di istanza i della sua Outer associata
12      void print() { System.out.println(i); }
13  }
14
15  public void outerMethod() {
16      Nested.print();
17      Inner innerObject = new Inner(); // Qui 'this.new Inner()' è implicito
18      innerObject.print();
19  }
20 }

```

0.1.3 1.3 Local Classes (Classi Locali a un Metodo)

Una classe definita all'interno del corpo di un blocco di codice (tipicamente un metodo):

```

1  public void outerMethod() {
2      final int j = 1;
3
4      class Local { // NESSUN modificatore di visibilità ammesso (no public, private, protected)
5          int addOne() { return j + 1; }
6      }
7
8      Local localObject = new Local();
9      System.out.println(localObject.addOne()); // Stampa 2
10 }

```

- **Regole di visibilità:** La classe è visibile e istanziabile **esclusivamente all'interno del blocco/metodo** in cui è dichiarata.
- **Cattura delle variabili locali (Variable Capture & Closures):**
 - Può accedere alle variabili locali e ai parametri del metodo contenitore **solo se sono dichiarati final oppure sono effectively final** (ovvero il loro valore non viene mai modificato dopo l'inizializzazione).
 - **Perché questo vincolo severo? (Domanda classica d'esame):**
 - * Le variabili locali risiedono nello **Stack Frame** del metodo e vengono distrutte non appena il metodo termina (return).
 - * L'oggetto della classe locale (localObject) risiede nell'**Heap** e potrebbe sopravvivere ben oltre la fine del metodo (ad esempio se restituito come interfaccia o registrato come listener).
 - * Per permettere all'oggetto di accedere a j, il compilatore Java crea un campo sintetico nascosto dentro Local e vi **copia** il valore di j al momento della creazione.
 - * Se j potesse essere riassegnato (j++), il valore nello stack e la copia nell'heap diverrebbero disallineati, rompendo la coerenza del linguaggio. Rendendo la variabile immutabile (final), la copia rimane identica e coerente per sempre.

0.1.4 1.4 Il Meme della Slide 10: *Objects inside objects*

Nella Slide 10, il prof. Pasqua inserisce il celebre meme di Xzibit (*Pimp My Ride*): > “YO DAWG, I HEAR YOU LIKE OBJECT ORIENTED PROGRAMMING SO I NESTED A CLASS INSIDE YOUR CLASS SO YOU CAN CREATE OBJECTS WHILE YOU CREATE OBJECTS”

A sottolineare con ironia la tentazione (e il potere) di creare gerarchie di classi dentro classi per incapsulare comportamenti strettamente accoppiati.

0.1.5 1.5 Motivazioni Architetture & Il Caso di Studio *Iterator Pattern* (Slide 11-12)

Perché annidare le classi? 1. **Migliore organizzazione del codice**: collocare classi di supporto logico all'interno del namespace della sola classe che le utilizza. 2. **Politica di visibilità privilegiata (*Privileged Access*)**: le classi annidate possono leggere lo stato privato della classe ospitante senza costringere a esporre metodi public o package-private pericolosi verso l'esterno. 3. **Meno complessità nel codebase**: evita la proliferazione di decine di minuscoli file `.java` separati.

L'Esempio del pattern *Iterable* / *Iterator* (Slide 12)

- **Senza annidamento (Due classi separate)**: `CRIterator` deve essere dichiarata all'esterno, riceve l'array `Student[] arr` nel costruttore e necessita di visibilità speciale:

```
1 class Classroom implements Iterable {
2     private Student[] students;
3     @Override
4     public Iterator iterator() { return new CRIterator(students); }
5 }
6 class CRIterator implements Iterator {
7     private Student[] arr;
8     private int next = 0;
9     CRIterator(Student[] arr) { this.arr = arr; }
10    @Override public boolean hasNext() { return next < arr.length; }
11    @Override public Object next() { return new Student(arr[next++]); }
12 }
```

- **Con Inner Class (Soluzione idiomatica Java)**: `CRIterator` è una Inner Class dentro `ClassRoom`. Non serve memorizzare un array duplicato: accede direttamente al campo privato `students` di `ClassRoom`!

```
1 class Classroom implements Iterable {
2     private Student[] students;
3     @Override
4     public Iterator iterator() { return new CRIterator(); }
5
6     class CRIterator implements Iterator {
7         private int next = 0;
8         @Override public boolean hasNext() { return next < students.length; }
9         @Override public Object next() { return new Student(students[next++]); }
10    }
11 }
```

0.1.6 1.6 Anonymous Classes (Classi Anonime, Slide 13-16)

Una classe locale **priva di nome**, definita e istanziata contestualmente all'interno di una singola espressione.

* **Vincolo**: Può essere creata solo estendendo una classe esistente o implementando un'interfaccia: “java // 1. Estendendo una classe esistente: `Person sam = new Person(“Sam”) { @Override public String toString() { return “It’s Sam!”; } }`;

// 2. Implementando un'interfaccia: `Time init = new Time() { @Override public int getHours() { return 0; } @Override public int getMinutes() { return 0; } @Override public int getSeconds() { return 0; } }`; *

****Regola aurea sui Costruttori****: **Una classe anonima NON può dichiarare alcun costruttore esplicito!** (Poiché i costruttori devono avere per definizione lo stesso nome della classe, e una classe anonima non ha nome). Eventuali inizializzazioni devono sfruttare gli **instance initializer blocks** `{ ... }` o i parametri passati al costruttore della superclasse (`new SuperClass(arg1, arg2){ ... }`). * ****Espressioni****: Essendo espressioni, devono terminare con un punto e virgola `;` (se assegnate a una variabile) o possono essere passate direttamente come parametri attuali di un metodo: `java tasks.add(new Runnable() { @Override public void run() { System.out.println(“Running!”); } });` “

0.1.7 1.7 Lambda Expressions (Slide 17-19)

Introdotte in Java 8, sono una notazione sintattica ultra-compatta per implementare **interfacce funzionali** (interfacce con un **unico metodo astratto**, note come SAM — *Single Abstract Method*).

```
1 (parametri) -> corpo
```

- **Parametri:**

- Nessun parametro: `() -> 42`
- Un parametro: `x -> x + 1` (le parentesi tonde sono opzionali se non si specifica il tipo esplicito)
- Due o più parametri: `(x, y) -> x + y`

- **Corpo:**

- Espressione singola: `param -> param - 1` (restituzione implicita del valore calcolato, niente `return` né graffe)
- Blocco di codice: `param -> { int res = param - 1; return res; }` (richiede graffe esplicite e `return`)

- **Inferenza di tipo (*Type Inference*):** Il compilatore Java deduce i tipi dei parametri e del valore di ritorno dal contesto d'uso (*target typing* dell'interfaccia funzionale).

- **Differenza tra Classi Anonime e Lambda:**

- La classe anonima può implementare interfacce con più metodi astratti o estendere classi concrete/astratte; la lambda può implementare **solo interfacce funzionali SAM**.
- La classe anonima crea un nuovo scope per `this` (che punta all'istanza anonima stessa); nella lambda `this` è **lessicale** (punta all'istanza della classe circostante).
- La classe anonima può dichiarare campi di istanza che mantengono stato; la lambda è per natura *stateless*.

0.1.8 1.8 Varargs (type... args, Slide 20-21)

Consente a un metodo di accettare un numero variabile (da zero a molti) di argomenti dello stesso tipo:

```
1 static int min(int... values) {
2     int res = Integer.MAX_VALUE;
3     for (int v : values)
4         if (v < res) res = v;
5     return res;
6 }
```

- **Dietro le quinte del compilatore:** Il parametro `int... values` viene tradotto esattamente in un array `int[] values`. Al momento della chiamata:

- `min(1, 0, 5)` → il compilatore inietta il codice: `min(new int[]{ 1, 0, 5 })`.
- `min()` (senza argomenti) → crea un array vuoto `min(new int[]{})`.

- **Regole tassative:**

1. Ci può essere **al massimo un** parametro `varargs` per metodo.
2. Il parametro `varargs` deve essere tassativamente l'**ultimo** nella lista dei parametri formali (`void foo(String s, int... nums)` è valido; `void foo(int... nums, String s)` genera errore di compilazione).

0.2 2. Analisi Dettagliata del Codice (Lezione14/MavenDate)

Nella Lezione 14 il progetto [MavenDate](#) introduce il primo ponte fondamentale verso i **Tipi Generici** (anticipando la teoria formale di Slide T15) e sfrutta le classi annidate, le classi anonime e le lambda viste nella Lezione 13.

0.2.1 2.1 Diff di `Date.java` (Lezione 13 vs Lezione 14)

Fino alla Lezione 13, `Date` implementava l'interfaccia non tipizzata (raw) `Comparable`:

```

1  --- Lezione13/Date.java
2  +++ Lezione14/Date.java
3  @@ -1,6 +1,6 @@
4  package it.oop.core;
5
6  -public class Date implements Comparable {
7  +public class Date implements Comparable<Date> {
8
9  @@ -73,15 +73,14 @@
10     @Override
11     - public int compareTo(Object other) {
12     -     Date otherAsDate = (Date) other;
13     -     int diff = this.year - otherAsDate.getYear();
14     + public int compareTo(Date other) {
15     +     int diff = this.year - other.getYear();
16     +     if (diff != 0)
17     +         return diff;
18     -     diff = this.month - otherAsDate.getMonth();
19     +     diff = this.month - other.getMonth();
20     +     if (diff != 0)
21     +         return diff;
22     -     return this.day - otherAsDate.getDay();
23     +     return this.day - other.getDay();
24     }

```

- **Beneficio immediato del Generics:**

1. La firma del metodo diventa fortemente tipizzata a compile-time: `compareTo(Date other)` anziché `compareTo(Object other)`.
2. Viene eliminato il cast esplicito `(Date)other`, azzerando il rischio di incorrere a runtime in una `ClassCastException`.

0.2.2 2.2 La Nuova Classe Generica `Pair.java`

```

1  package it.oop.core;
2
3  public class Pair<F, S> {
4      private final F first;
5      private final S second;
6
7      public Pair(F first, S second) {
8          this.first = first;
9          this.second = second;
10     }
11
12     public F getFirst() { return first; }
13     public S getSecond() { return second; }
14 }

```

- *F* (*First*) e *s* (*Second*) sono due **Type Parameters** indipendenti.
- I campi sono `private final`, rendendo la coppia immutabile una volta istanziata.

0.2.3 2.3 La Classe Generica con Bounded Type `OrderedPair.java`

```

1 package it.oop.core;
2
3 public class OrderedPair <T extends Comparable<T>> {
4     private final T first;
5     private final T second;
6
7     public OrderedPair(T first, T second) {
8         this.first = first;
9         this.second = second;
10        if (first.compareTo(second) > 0)
11            System.out.println("Pair not ordered!");
12    }
13
14    public T getFirst() { return first; }
15    public T getSecond() { return second; }
16 }

```

- **Bounded Type Parameter** (`<T extends Comparable<T>>`):
 - Il tipo `T` non può essere un qualsiasi tipo `Object`: deve tassativamente implementare l'interfaccia `Comparable<T>`.
 - Grazie a questo vincolo, il compilatore consente all'interno del costruttore di invocare legalmente `first.compareTo(second)`.
 - Se si tentasse di istanziare `OrderedPair<Object>`, il compilatore bloccherebbe l'operazione con un errore prima ancora di eseguire.

0.2.4 2.4 Le Sottoclassi Specializzate `DatePair.java` e `DateInterval.java`

`DatePair.java`

```

1 package it.oop.core;
2
3 public class DatePair extends Pair<Date, Date> {
4     public DatePair(Date left, Date right) {
5         super(left, right);
6     }
7
8     public Date getLeft() { return getFirst(); }
9     public Date getRight() { return getSecond(); }
10
11    @Override
12    public String toString() {
13        Date l = getLeft();
14        Date r = getRight();
15        return String.format("[%s .. %s]", l.toString(), r.toString());
16    }
17 }

```

- Fissa sia *F* che *s* al tipo concreto `Date` (`extends Pair<Date, Date>`).
- Offre metodi con semantica di dominio più chiara: `getLeft()` e `getRight()`.

DateInterval.java

```

1 package it.oop.core;
2
3 public class DateInterval extends OrderedPair<Date> {
4     public DateInterval(Date left, Date right) {
5         super(left, right);
6     }
7
8     public Date getLeft() { return getFirst(); }
9     public Date getRight() { return getSecond(); }
10
11     @Override
12     public String toString() {
13         Date l = getLeft();
14         Date r = getRight();
15         return String.format("[%s .. %s]", l.toString(), r.toString());
16     }
17 }

```

- Eredita da `OrderedPair<Date>`: è legale perché `Date` implements `Comparable<Date>`.
- Se le date passate non sono cronologicamente ordinate (`left > right`), il costruttore di `OrderedPair` emette il messaggio d'avviso "Pair not ordered!".

0.2.5 2.5 MainDate.java

Raccoglie in un unico punto di ingresso tutte le feature viste nelle lezioni 13 e 14:

```

1 package it.oop.ui;
2
3 import it.oop.core.*;
4 import it.oop.core.Date;
5
6 public class MainDate {
7
8     public static void main(String[] args) {
9         // 1. Static Nested Class (Builder Pattern da Lezione 13)
10        Date.Builder dateBuilder = new Date.Builder(2025);
11        Date d1 = dateBuilder.build(10,9);
12        Date d2 = dateBuilder.build(10,-1);
13        System.out.println(d1.toString());
14        System.out.println(d2.toString());
15
16        // 2. Anonymous Class che implementa l'interfaccia Time (Lezione 13/14)
17        Time init = new Time() {
18            @Override public int getHours() { return 0; }
19            @Override public int getMinutes() { return 0; }
20            @Override public int getSeconds() { return 0; }
21            @Override public String toString() {
22                return String.format("%02d:%02d:%02d", getHours(), getMinutes(), getSeconds());
23            }
24        };
25        System.out.println(init.toString());
26
27        // 3. Lambda Expression che implementa FormattedDateConverter (Lezione 13/14)
28        FormattedDateConverter toAmerican =
29            d -> new AmericanDate(d.getDay(), d.getMonth(), d.getYear());
30        System.out.println(toAmerican.convert(new ItalianDate(11, 11, 2025)) instanceof
31            AmericanDate);
32
33        // 4. Generics: Pair con Diamond Operator <>
34        Pair<String, FormattedDate> event =

```



```

34         new Pair<>("OOP exam", new ItalianDate(2, 2, 2026));
35         System.out.println(event.getFirst() + " on " + event.getSecond().prettyPrint());
36
37         // 5. DatePair e DateInterval
38         DatePair dp = new DatePair(new Date(1,2,2025), new Date(1,1,2025));
39         System.out.println("date pair: " + dp.toString());
40
41         // Qui scatta l'avviso di ordinamento perché 1/2/2025 è successivo a 1/1/2025!
42         DateInterval di = new DateInterval(new Date(1,2,2025), new Date(1,1,2025));
43
44         // 6. Wildcard con Bounded Type (anticipazione Lezione 15)
45         Pair<?, ? extends FormattedDate> unknown = event;
46         System.out.println(unknown.getFirst().toString() + " on " +
47             unknown.getSecond().prettyPrint());
48     }

```

0.3 3. Risultato di Compilazione ed Esecuzione Reale

Comando eseguito nel workspace del corso:

```
1 mvn compile exec:java -Dexec.mainClass="it.oop.ui.MainDate"
```

Output ottenuto:

```

1 y2025m9d10
2 y2025m1d1
3 00:00:00
4 true
5 OOP exam on 2 febbraio 2026
6 date pair: [y2025m2d1 .. y2025m1d1]
7 Pair not ordered!
8 OOP exam on 2 febbraio 2026

```

0.3.1 Dettaglio riga per riga dell'output:

1. y2025m9d10 → generato da d1 = dateBuilder.build(10, 9) (10 settembre 2025).
2. y2025m1d1 → generato da d2 = dateBuilder.build(10, -1) (mese errato fallback su 1/1/2025).
3. 00:00:00 → generato dall'override di toString() della **classe anonima** init che implementa Time.
4. true → la **lambda** toAmerican converte l'istanza di ItalianDate in AmericanDate.
5. OOP exam on 2 febbraio 2026 → Pair<String, FormattedDate> con formattazione italiana.
6. date pair: [y2025m2d1 .. y2025m1d1] → DatePair stampa le due date senza controllo di sequenzialità.
7. Pair not ordered! → emesso dal costruttore di OrderedPair dentro new DateInterval(1/2/2025, 1/1/2025) perché 1/2/2025 > 1/1/2025.
8. OOP exam on 2 febbraio 2026 → invocazione polimorfica tramite wildcard generica Pair<?, ? extends FormattedDate>.

Dimmi “vai” per procedere con il **Blocco 14** (Lezione 15 / Slide T15 — *Generic Types, Type Erasure, Wildcards, PECS e diff in Lezione15/MavenDate*).